Web Scraping using Python

Design Laboratory (CS69202)
Feb 4, 2026

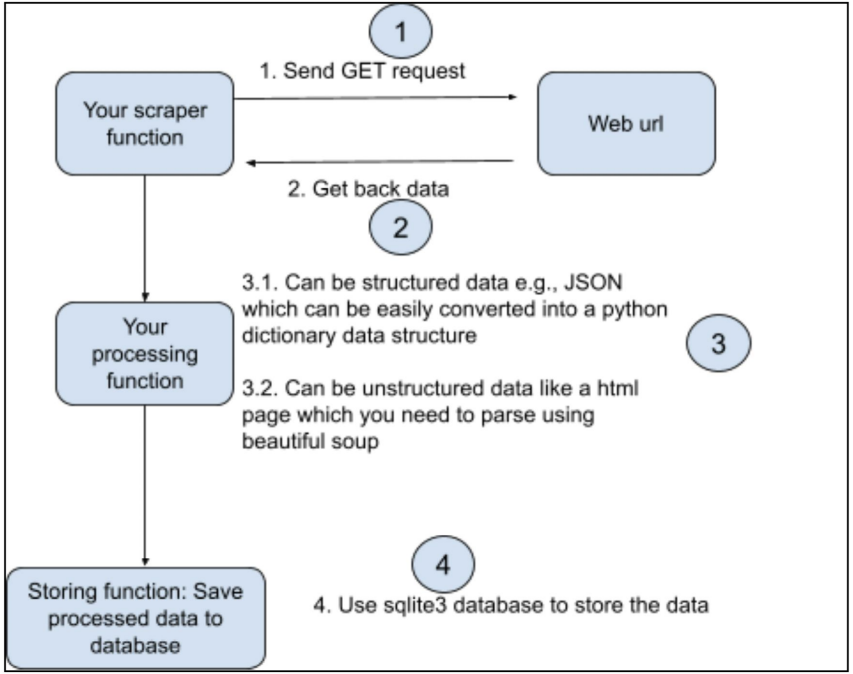
What is Web Scrapping

Web scrapers are programs that can collect data from websites, parse it and store the data (e.g., for analysis later).

A web request can be GET or POST. Using GET requests you can get data from the world wide web (WWW).

Pipeline for Scraping

1. Send the request (GET/POST)
2. Get the response
3. Based on response type; process the data
4. Store the data



Tools we will be learning

1. **Urllib:** “A built-in Python library used to fetch URLs and handle low-level HTTP requests, responses, headers, and cookies.”
2. **Requests**: “allows you to send HTTP/1.1 requests extremely easily”
3. **BeautifulSoup (BS4):** “scrape information from web pages. Sits atop an HTML parser, assists iterating, searching, and modifying the parse tree.”
4. **Selenium**: “Primarily it is for automating web applications for testing purposes, but is certainly not limited to just that.”
5. **Sqlite3**: “provides a lightweight disk-based database that doesn’t require a separate server process and allows accessing the database using a nonstandard variant of the SQL query language.”

What is urllib?

1. urllib is a standard Python library for:
    a. Fetching URLs
    b. Sending HTTP/HTTPS requests
    c. Handling responses
2. Does not require installation
3. Lower-level than requests, but more transparent

Main submodules:

- **urllib.request** – open URLs
- **urllib.parse** – URL encoding/decoding
- **urllib.error** – error handling

Urllib - parse

```
from urllib.parse import urlparse
urlparse("scheme://netloc/path;parameters?query#fragment")
o =
urlparse("http://docs.python.org:80/3/library/urllib.parse.html?"
             "highlight=params#url-parsing")
print(o)
print(o.scheme)
print(o.netloc)
print(o.hostname)
print(o.port)
print(o._replace(fragment="").geturl())
```

```
ParseResult(scheme='http', netloc='docs.python.org:80', path='/3/library/urllib.parse.html', params='',
query='highlight=params', fragment='url-parsing')
http
docs.python.org:80
docs.python.org
80
http://docs.python.org:80/3/library/urllib.parse.html?highlight=params
```

Urllib - request

```
import urllib.request
req = urllib.request.Request('http://www.example.com/')
req.add_header('Referer', 'http://www.python.org/')
# Customize the default User-Agent header value:
req.add_header('User-Agent', 'urllib-example/0.1 (Contact: . .
.)')
with urllib.request.urlopen(req) as f:
    print(f.read().decode('utf-8'))
```

```
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport"
content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;
font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</
style><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without
needing permission. Avoid use in operations.<p><a href="https://iana.org/domains/example">Learn more</
a></div></body></html>
```

Requests Library

1. requests is a high-level HTTP library in Python
2. Simplifies sending HTTP requests compared to urllib
3. Handles headers, cookies, sessions, and authentication easily

Key features:

- Simple and readable syntax
- Automatic JSON handling
- Built-in cookie & session management
- Widely used in industry and research

Requests Library - Example

1. **Sending a GET request:**

```
response = requests.get("https://httpbin.org/get")
print(response.text)
```

2. **Sending headers with requests:**

```
headers = {"User-Agent": "Mozilla/5.0",
    "Accept": "application/json"}
response = requests.get("https://httpbin.org/headers", headers=headers)
print(response.json())
```

BeautifulSoup

```
html_doc = """
<html><head><title>The Dormouse's story</title></head>
<body>
<p class="title"><b>The Dormouse's story</b></p>

<p class="story">Once upon a time there were three little sisters; and their names were
<a href="http://example.com/elsie" class="sister" id="link1">Elsie</a>,
<a href="http://example.com/lacie" class="sister" id="link2">Lacie</a> and
<a href="http://example.com/tillie" class="sister" id="link3">Tillie</a>;
and they lived at the bottom of a well.</p>

<p class="story">...</p>
"""
```

```
from bs4 import BeautifulSoup

soup = BeautifulSoup(html_doc, 'html.parser')
```

Here are some simple ways to navigate that data structure:

```
soup.title
# <title>The Dormouse's story</title>

soup.title.name
# u'title'

soup.title.string
# u'The Dormouse's story'

soup.title.parent.name
# u'head'

soup.p
# <p class="title">><b>The Dormouse's story</b></p>

soup.p['class']
# u'title'

soup.a
# <a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>

soup.find_all('a')
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
#  <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
#  <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]

soup.find(id="link3")
# <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>
```

BeautifulSoup: Searching the tree

Two most popular methods: find() and find_all()

You can search for an element using different kinds of filters:

- `String: soup.find_all('b')`
- `Regex : soup.find_all(re.compile("^b"))`
- `List  : soup.find_all(["a", "b"])`

find_all(name, attrs, recursive, string, limit, **kwargs)

```
soup.find_all("title")
# [<title>The Dormouse's story</title>]

soup.find_all("p", "title")
# [<p class="title"><b>The Dormouse's story</b></p>]

soup.find_all("a")
# [<a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>,
#  <a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>,
#  <a class="sister" href="http://example.com/tillie" id="link3">Tillie</a>]

soup.find_all(id="link2")
# [<a class="sister" href="http://example.com/lacie" id="link2">Lacie</a>]

import re
soup.find(string=re.compile("sisters"))
# u'Once upon a time there were three little sisters; and their names were\n'
```

find_all(name, attrs, recursive, string, limit, **kwargs)

1. Find tags starting with specific letters:
   a. **soup.find_all(re.compile("^h")) -** Matches: <html>, <head>, <h1>, <h2>
2. Find multiple tag types:
   a. **soup.find_all(re.compile("div|span"))**
3. Match class names starting with product:
   a. **soup.find_all("div", class_=re.compile("^product-"))**
4. Match IDs ending with numbers”
   a. **soup.find_all(id=re.compile(r"\d+$"))**
5. Find all HTTPS links
   a. **soup.find_all("a", href=re.compile("^https://"))**
6. Exclude certain links (negative regex)
   a. **soup.find_all("a", href=re.compile("^(?!.*logout)"))**

find(name, attrs, recursive, string, **kwargs)

The *find_all()* method scans the entire document looking for results, but sometimes you only want to find one result. — use **find()**

```
soup.find_all('title', limit=1)
# [<title>The Dormouse's story</title>]

soup.find('title')
# <title>The Dormouse's story</title>
```

find_all_next, find_next

These methods use iterates over whatever tags and strings that come after it in the document

```
first_link = soup.a
first_link
# <a class="sister" href="http://example.com/elsie" id="link1">Elsie</a>

first_link.find_all_next(string=True)
# [u'Elsie', u',\n', u'Lacie', u' and\n', u'Tillie',
#  u';\nand they lived at the bottom of a well.', u'\n\n', u'...', u'\n']

first_link.find_next("p")
# <p class="story">...</p>
```

Selenium: Writing your first script

- Start the session
  ```
  driver = webdriver.Chrome()
  ```
- Take action on browser
  ```
  driver.get("https://www.selenium.dev/selenium/web/web-form.html")
  ```
- Request browser information
  ```
  title = driver.title
  ```
- Find an element
  ```
  text_box = driver.find_element(by=By.NAME, value="my-text")
  submit_button = driver.find_element(by=By.CSS_SELECTOR, value="button")
  ```

Selenium: Writing your first script

- Take action on element

```
text_box.send_keys("Selenium")
submit_button.click()
```

- Request element information

```
text = message.text
```

- End the session

```
driver.quit()
```

Selenium: Finding Elements Using XPath

1. **Basic XPath selection:**

   ```
   element = driver.find_element(By.XPATH, "//input[@name='username']")
   ```

2. **XPath using contains():**

   ```
   driver.find_element(By.XPATH, "//button[contains(text(),'Submit')]")
   ```

3. **Select element by partial class name:**

   ```
   driver.find_element(By.XPATH, "//div[contains(@class,'product')]")
   ```

4. **Select nth element:**

   ```
   driver.find_element(By.XPATH, "(//input[@type='text'])[2]")
   ```

5. **Select child element:**

   ```
   driver.find_element(By.XPATH, "//div[@id='menu']/a[1]")
   ```

Selenium: Clicking and Sending Input

1. **Send input to text box:**

```
textbox = driver.find_element(By.XPATH, "//input[@id='email']")
textbox.send_keys("test@example.com")
```

2. **Click a button:**

```
button =
driver.find_element(By.XPATH,"//button[@type='submit']")
button.click()
```

Sqlite3: Writing your first script

- Create a new database and open a database connection
```
import sqlite3
con = sqlite3.connect("tutorial.db")
```
- Create a database cursor: to execute SQL statements and fetch results from SQL queries
```
cur = con.cursor()
```
- Create a database table movie with columns for title, release year, and review score
```
cur.execute("CREATE TABLE movie(title, year, score)")
```
- Add data supplied as SQL literals by executing an INSERT statement
```
cur.execute("""
     INSERT INTO movie VALUES
     ('Monty Python and the Holy Grail', 1975, 8.2)
""")
```

Sqlite3: Writing your first script

- Fetching results from a table by executing SELECT
```
res = cur.execute("SELECT score FROM movie")
res.fetchall()
```
- Saving the changes
```
con.commit()
```
- Closing the db connection
```
con.close()
```